Centro Universitário de Excelência
Sistemas de Informação

OAT 2 – Monitorando a Continuidade

Escalabilidade e IoT: monitoramento, alarmes e HPA

Autores: Joaqson Rodrigues Miranda
Lucas Santos Oliveira
Manoel Souza Santos
Monique Prado Pereira Gomes
Raian Rocha Santos

Vitória da Conquista, 2026

Agenda

Mostrar, com evidências, que o cluster deixou de ser uma caixa preta: agora é monitorado, emite alarmes e escala sozinho.

1. Contexto:
Problema e objetivos

2. Dados:
Simulador IoT e Prometheus

3. Desenvolvimento:
Dashboards, alarmes e HPA

4. Estresse:
Evidências do teste

5. Conclusão:
SLA, Runbooks e resultados

Contexto do problema

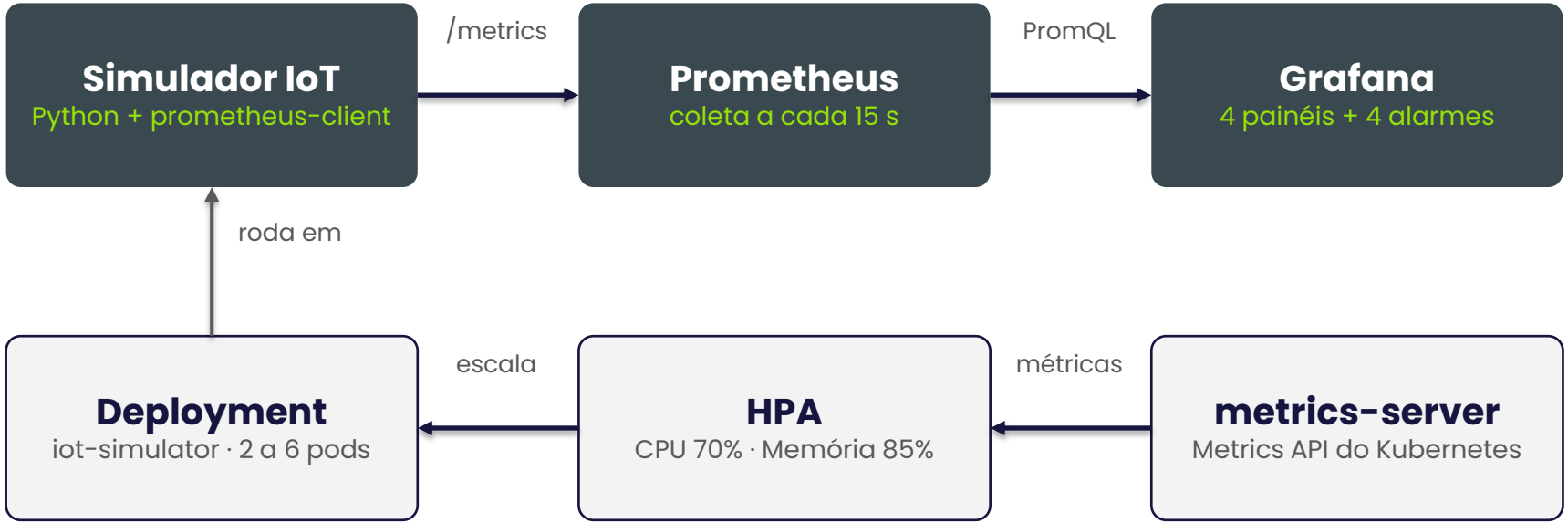
- **Picos de IoT:** milhares de leituras de sensores às 08h e 17h.
- **Caixa preta:** a Engenharia não enxerga a saúde das aplicações.
- **Contrato Enterprise:** disponibilidade de 99,9%.
- **Missão:** observar, alertar e escalar sozinho.

99,9% disponibilidade exigida pelos clientes Enterprise

~1.000/s leituras IoT simuladas pelo gerador de carga

4 + 4 painéis e alarmes obrigatórios, mais o HPA

Arquitetura da solução



Ambiente: Kubernetes local (Docker Desktop) · manifestos na pasta k8s/ do repositório

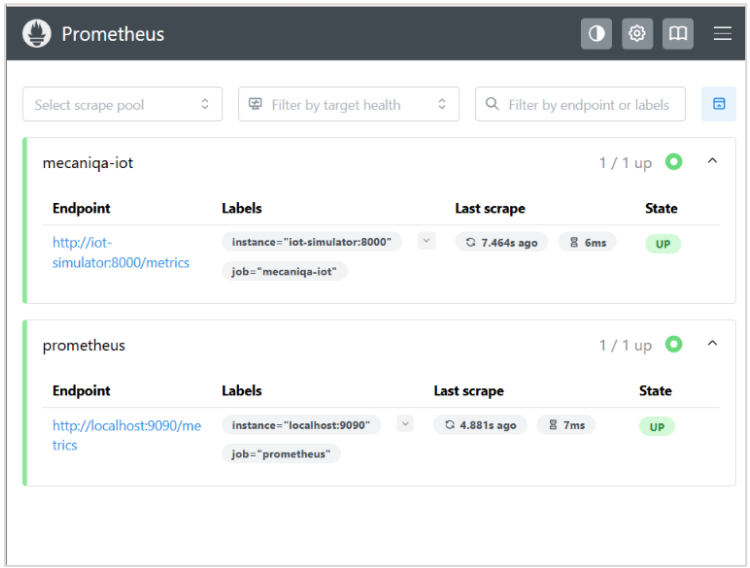
Dados e instrumentação

```python
from prometheus_client import Counter, Histogram

READING_RESULTS = Counter(
    "mecaniqa_iot_reading_results_total",
    "...", ["result"])
DIAGNOSTICS_SUCCESS = Counter(
    "mecaniqa_iot_diagnostics_success_total", "...")
DIAGNOSTICS_ERRORS = Counter(
    "mecaniqa_iot_diagnostics_errors_total", "...")
PROCESSING_TIME = Histogram(
    "mecaniqa_iot_batch_processing_seconds", "...")

start_http_server(8000)  # expõe /metrics
```

iot-simulator/simulator.py (trecho)



Prometheus > Targets: simulador e Prometheus em UP

Dashboards no Grafana

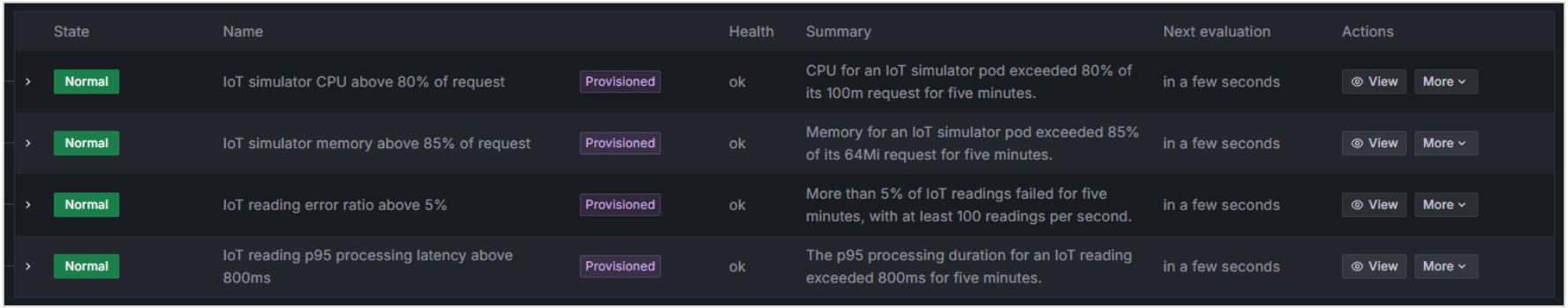








Alarmes configurados



State	Name		Health	Summary	Next evaluation	Actions
Normal	IoT simulator CPU above 80% of request	Provisioned	ok	CPU for an IoT simulator pod exceeded 80% of its 100m request for five minutes.	in a few seconds	View More
Normal	IoT simulator memory above 85% of request	Provisioned	ok	Memory for an IoT simulator pod exceeded 85% of its 64Mi request for five minutes.	in a few seconds	View More
Normal	IoT reading error ratio above 5%	Provisioned	ok	More than 5% of IoT readings failed for five minutes, with at least 100 readings per second.	in a few seconds	View More
Normal	IoT reading p95 processing latency above 800ms	Provisioned	ok	The p95 processing duration for an IoT reading exceeded 800ms for five minutes.	in a few seconds	View More

CPU
> 80%
do request (100m)

Memória
> 85%
do request (64 MiB)

Erros
> 5%
das leituras, com > 100/s

Latência
p95 > 800 ms
tempo de processamento

4 regras provisionadas (k8s/grafana-alerting.yaml) - janela de 5 min com persistência de 5 min para evitar falsos positivos

HPA: autoscaling

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: iot-simulator
spec:
  minReplicas: 2
  maxReplicas: 6
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          averageUtilization: 70
    # memory → averageUtilization: 85
```

k8s/iot-simulator-hpa.yaml (trecho)

2 a 6 réplicas

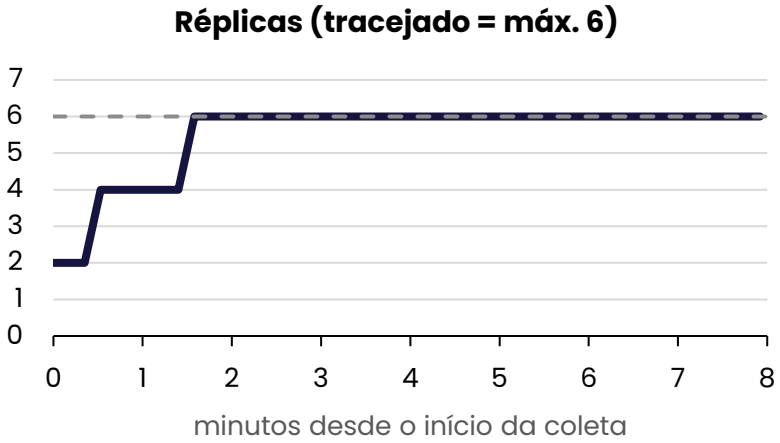
70% meta de CPU

85% meta de memória

```
NAME           REFERENCE                     TARGETS                       MINPODS   MAXPODS   REPLICAS   AGE
iot-simulator  Deployment/iot-simulator      cpu: 1%/70%, memory: 22%/85%  2         6         6          41m
```

kubectl get hpa: 6/6 réplicas após a carga

Estresse: HPA escalando



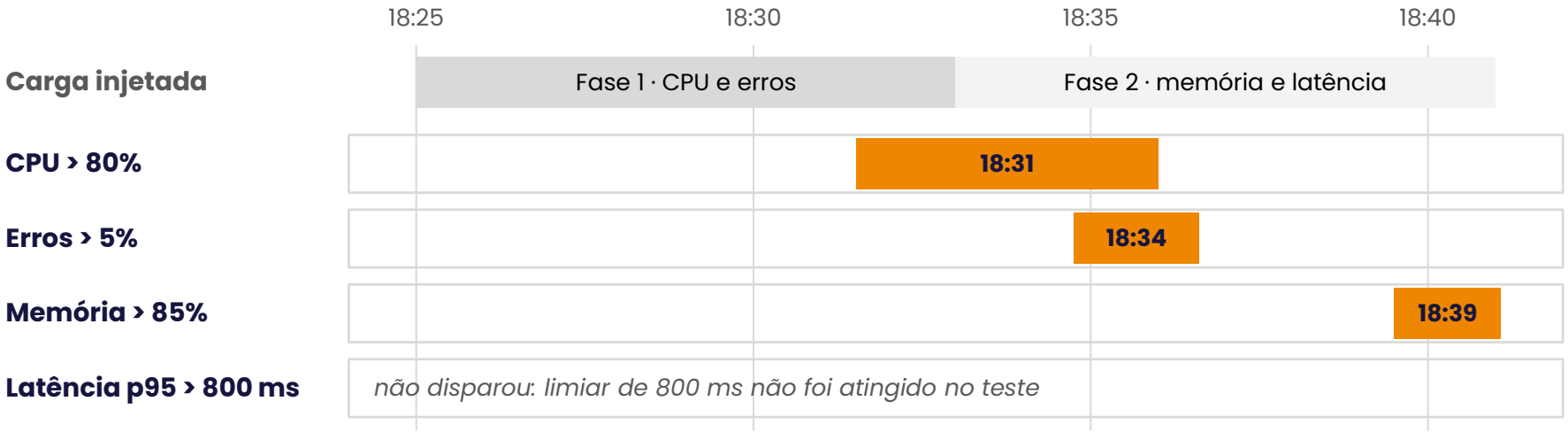


Prometheus: CPU por pod no pico de carga (execução das 18:55)

```
18:25:11  réplicas: 2   CPU/pod ≈  20m
18:25:32  réplicas: 4   CPU/pod ≈ 258m
18:26:35  réplicas: 6   CPU/pod ≈ 168m  (máximo)
```

Fonte: evidencias/stress-20261113-182435/cpu-memory-errors-top-pods.log

Estresse: alarmes




```
18:31:31  active  IoT simulator CPU above 80% of request
18:34:45  active  IoT reading error ratio above 5%
18:39:30  active  IoT simulator memory above 85% of request
```

Fonte: evidencias/stress-20261113-182435/grafana-alert-states.jsonl (estados capturados a cada 30 s)

Continuidade do negócio

SLI	Leituras com sucesso ÷ leituras elegíveis
SLO	99,95% de disponibilidade mensal
SLA	99,9% mensal (≈ 43 min de queda/mês)
RTO	até 60 min para restaurar o serviço
RPO	até 5 min de leituras perdidas

Detalhamento em docs/observabilidade-e-continuidade.md

Runbooks por alarme

CPU	**Memória**
Erros	**Latência**

1. Confirmar o sinal
2. Diagnosticar com kubectl
3. Mitigar (carga, scale, rollback)
4. Validar a recuperação

Considerações finais

- ✓ 4 dashboards no Grafana
- ✓ 4 alarmes ativos e testados
- ✓ HPA de 2 a 6 réplicas sob carga
- ✓ Logs e capturas do estresse
- ✓ SLA/SLO/SLI, RTO/RPO e Runbooks

Limites e próximos passos

- Latência p95 não disparou: calibrar o teste.
- Prometheus e Grafana sem volume persistente.
- RTO/RPO ainda são metas: falta failover multi-região.

Referências

- **Repositório:** github.com/Raianrocha/mecaniQA-nuvem-fortaleza
- **Documentação da equipe:** docs/observabilidade-e-continuidade.md
- **Prometheus:** prometheus.io/docs
- **Grafana Alerting:** grafana.com/docs/grafana/latest/alerting
- **Kubernetes HPA:** kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale
- **SLI e SLO (Google SRE):** sre.google/sre-book/service-level-objectives